KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

R E S U L T R E P O R T

마이크로프로세서응용및실험 결과보고서_인터럽트



수 강 과 목	: 마이크로프로세서응용및실험-2분반
담 당 교 수	: 김도인 교수님
학 번	: 202210820
작 성 자	: 이동제
공 동 실 험 자	: 강민수(202214221)
제 출 일	: 2024-05-09 목요일

<목 차>

1. 실험목적 ····· 2

2. 실험원리 ····· 2

3. 실험기구 및 재료 ····· 2

4. 실험방법 ····· 3-5

5. 측정값 ····· 6

6. 실험결과 ····· 6

7. 결과에 대한 논의 ····· 7

8. 결론 ····· 7

9. 참고문헌 및 출처 ····· 7

1. 실험목적

Interrupt 처리 방법을 익힌다. Interrupt를 통해 기존 동작 도중에도 다른 동작을 할 수 있다.

2. 실험원리

지금 ATmega128 마이크로 컨트롤러로 사용할 수 있는 물리적 인터럽트 발생 장치는 INT0, INT1에 연결된 스위치 두 개를 가지고 테스트합니다.

ATmega128은 35개의 인터럽트 소스가 있습니다. 그중 포트 B에 해당되는 핀만 사용할 것입니다.

인터럽트는 프로그램 작동하던 중에도 다른 작업이 발생하여 수행되는 것을 의미합니다.
마치 책을 읽던 상황에서 전화벨이 울리면 책 읽는 것을 멈추고 전화를 받아야 하는 것과 같습니다.

신호의 종류는 Falling edge, Rising edge, Low level 총 3가지로 EICRA의 해당되는 비트를 0과 1로 바꾸어서 원하는 종류의 신호를 사용할 수 있습니다.

인터럽트 서비스 루틴(인터럽트의 처리를 위해 미리 만들어진 프로그램)을 통해 인터럽트가 발생할 때 실행되는 함수를 사용합니다. ex) ISR(인터럽트 벡터), ISR(INT0_vect)

인터럽트 벡터란 외부 인터럽트 0, 외부 인터럽트 1에 해당되는 서비스 루틴이 실행되는 시작 주소입니다.
오늘 사용할 인터럽트는 RESET 다음으로 우선순위가 가종 높은 INT0과 INT1을 사용할 것이고, 시작 주소는 각각 INT0_vect, INT1_vect 입니다

이러한 실험 원리들을 가지고 Interrupt에 입력된 값에 따라 특정 인터럽트 값이 들어올 때, LED를 On/Off 하는 테스트를 진행합니다.

3. 실험기구 및 재료

ATmega128 마이크로 컨트롤러	1개
컴퓨터	1개
ISP 케이블	1개

표 1) 실험기구 및 재료

4. 실험방법

Interrupt을 동작시키는 테스트를 진행하기 위해 AVR Studio 속에서 코드를 작성합니다.
이번 실습교재6의 예제를 통해, 외부 인터럽트 INT0이 발생한 경우 상위 4개의 LED를 On시키고, 외부 인터럽트 INT1이 발생한 경우 하위 4개 LED를 On하는 코드로 정상 작동하는 것을 확인하였습니다.

후에 구술할 실험방법은 추가로 진행된 실습에 대해 작성했습니다.

1. 인터럽트 감지(Sensing) 방식에 따른 인터럽트의 발생 시점을 비교하기.

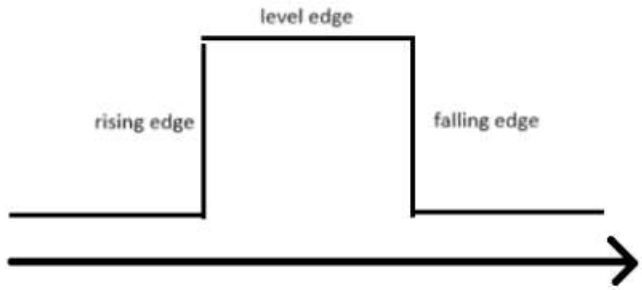


그림 1) 클럭 신호 (Clock signal)

EICRA에 각각 다른 값을 넣어 인터럽트 감지 방식을 바꿔보며 총 3가지 발생 시점을 테스트했습니다.

Low 레벨에서 인터럽트 발생 (EICRA = 0x00):
인터럽트는 해당 핀의 신호가 Low 레벨로 유지되는 동안 지속해서 발생합니다.
아무것도 누르지 않은 상태일 때 지속적으로 인터럽트가 발생했습니다.

하강 엣지에서 인터럽트 발생 (EICRA = 0x0A):
인터럽트는 해당 핀의 신호가 High에서 Low로 변경되는 falling edge에서 발생합니다.
버튼을 눌렀다가 떼는 순간부터 인터럽트가 발생했습니다.

상승 엣지에서 인터럽트 발생 (EICRA = 0x0F):
인터럽트는 해당 핀의 신호가 Low에서 High로 변경되는 rising edge에서 발생합니다.
버튼을 누르는 순간에 인터럽트가 발생했습니다.

2. 외부 인터럽트 INT0이 발생한 경우 LED0 → LED2 → LED4 → LED6 → LED1 → LED3 → LED5 → LED7 → LED0 → …를 순차적으로 On하고, 외부 인터럽트 INT1이 발생한 경우 LED를 멈추는 프로그램을 만드는 과정을 기록하기. (인터럽트의 발생 시점은 하강 에지로 설정함)

```
#define F_CPU 16000000L
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
```

기본값으로 CPU의 클럭 주파수를 16MHz로 설정시키고, 포트 및 레지스터에 대한 정의를 시키고, 지연

(Delay) 함수와 인터럽트(interrupt) 함수를 사용하기 위해서 AVR 라이브러리 속 헤더 파일을 포함하는 코드를 입력합니다.

```
int a = 0;
char current_pattern_value;
char next_pattern_value = 0b11111111;
char reverse_next_pattern_value = 0b11111101;
```

a라는 변수는 0이라는 값을 가지도록 합니다. 그리고 저번 오름차순 내림차순에서 썼던 current_pattern_value라는 변수를 쓸 것이라고 가정하고, next_pattern_value는 0b11111111이라는 값을 가지고, reverse_next_pattern_value는 0b11111101이라는 값을 가진 변수로 만듭니다.

```
ISR(INT0_vect)
{
        a=1;
        next_pattern_value = 0b11111110;
        reverse_next_pattern_value = 0b11111101;

}

ISR(INT1_vect)
{
        a=2;
}
```

ISR 함수를 통해 INT0에는 스위치1을 누를시, a 변수에 1이라는 값을 넣어주고, next_pattern_value를 초기값 0b11111110로 바꾸어서 메인 프로그램이 작동되게끔 만들어야 합니다. 그리고 INT1에는 a 변수에 2라는 값을 넣어주어 메인 프로그램 속에서 작동되도록 만듭니다.

```
int main(void)
{

        DDRB = 0xFF;      // 포트 B 핀을 출력으로 설정
        SREG |=0x80;
        EIMSK = 0x03;
        EICRA= 0x0A;
```

이제 인터럽트가 발생해도 다른 인터럽트에 반응할 수 있도록 인터럽트가 움직일 메인 동작을 만들기 위해 main(void)함수를 시작합니다. DDRB를 0xFF(0b11111111)로 설정하여 포트 B의 8개 핀을 출력으로 설정되게 만들고, SREG에 (|=)비트 Or 연산자를 이용하고, EIMSK를 통해 2개의 외부 인터럽트를 허용시킵니다.
EICRA를 0x0A(0b00001010)으로 설정하여 외부 인터럽트의 상승 엣지와 하강 엣지를 결정하는 비트인 ISCn1, ISCn0에 대해 값을 정해줍니다. ISCn0에 0, ISCn1엔 1을 입력하여 falling edge(하강 엣지)를 사용했습니다.

```
        if(a==2){
                while(1){
                        }
        }
```

메인 프로그램 내에 if문을 통해 a가 2일 때 작동하는 프로그램을 만듭니다. 이것은 INT1이 발생했을 때 작동하는 코드로 while문을 통해 프로그램이 진행되고 있는 자리에서 멈추게 만듭니다.

```
        while (1)
        {
                current_pattern_value = next_pattern_value;
```

```
PORTB = current_pattern_value;
```

그리고 current_pattern_value에 next_pattern_value를 집어넣고, 그 값을 PORTB에 넣습니다.

```
        // 다음 패턴 값 생성
    if(a==1){
    next_pattern_value = next_pattern_value << 2;
    next_pattern_value = next_pattern_value + 3;
    if(next_pattern_value == 0b11111111){
    a=-1;}
    }
```

a가 1일 때 발생되는 IF문을 통해 LED를 2비트씩 쉬프트시킵니다. 그리고 비트 쉬프트 연산을 통해 이동된 비트 위치에 0이 생기는 것을 방지하기 위해 쉬프트 할 때마다 3(0x03)을 더해줍니다.
이러한 동작을 반복하다가 최대값인 0xFF(0b11111111)에 도달했을 때 a에 -1이란 변수를 넣어줍니다.

```
    if(a==-1) {
    next_pattern_value = reverse_next_pattern_value;
    reverse_next_pattern_value = reverse_next_pattern_value << 2;
    reverse_next_pattern_value = reverse_next_pattern_value + 0b00000011;
```

위에서 a를 -1로 바꾸면 작동하는 IF문을 만들어줍니다. 짝수번째 작동에서 홀수번째 작동을 따로 지정해주기 위해 next_pattern_value를 reverse_next_pattern_value라는 변수를 통해 작동하도록 합니다.
그리고 똑같이 2비트씩 쉬프트시키며 이번엔 숫자 2(0b00000011)를 더해주어 다른 LED가 켜지지 않도록 합니다.

```
        if(next_pattern_value == 0b11111111){
            a=1;
            next_pattern_value = 0b11111110;
            reverse_next_pattern_value = 0b11111101;
                }
            }
```

최대값인 0b11111111에 도달하게 되면 a 변수에 1을 넣어주어 짝수패턴으로 돌아가서 반복동작하게끔 만듭니다. 그리고 next_pattern_value와 reverse_next_pattern_value의 값을 초기값으로 다시 설정해줘서 입력된 IF프로그램이 처음과 같이 정상 작동할 수 있도록 만들어줍니다.

```
        _delay_ms(300);
        }

    return 0;
}
```

마지막으로 딜레이 함수를 통해 300ms 동안 지연하게 하고, return을 통해 main함수를 종료시키고 0을 반환하게 만듭니다.

이렇게 AVR Studio에서 작성한 Interuppt code를 hex 파일로 저장하고 ISP 케이블을 ATmega128 마이크로컨트롤러에 연결합니다. 그리고 hex 파일을 실행시켜서 LED가 코드에서 요구하는 것과 같이 작동하는지 테스트합니다.

5. 측정값

INT0과 INT1에 각각 다른 값을 넣어 인터럽트 발생 시 다른 동작을 하도록 코드를 작성했다.

INT0	LED0 → LED2 → LED4 → LED6 → LED1 → LED3 → LED5 → LED7 → LED0 → …를 점등시키는 동작을 반복
INT1	INT0을 통해 진행 중이던 동작을 INT1이 발생한 시점에서 LED를 멈춤

표 2) AVR Studio를 통해 작성한 Interrupt 함수. 코딩 테스트 측정값

6. 실험결과

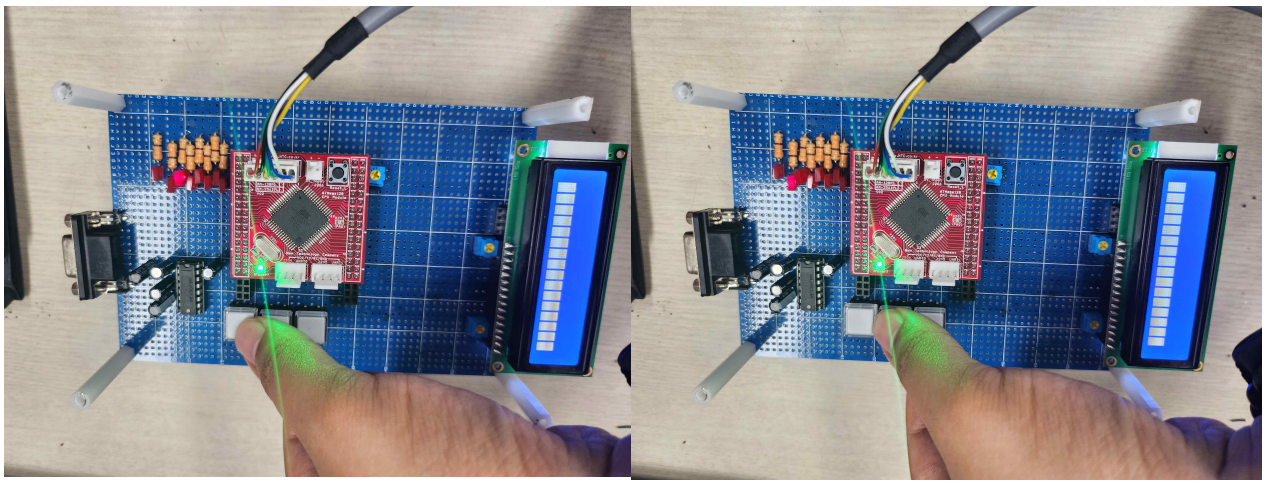


그림 2,3) 스위치를 통해 Interrupt 발생

작성한 코드가 스위치 동작을 통한 인터럽트 발생 시, LED가 정상적으로 작동하는지 확인하는 테스트를 진행하였다.

첫 번째 테스트, 2칸씩 LED를 쉬프트시켜 점등시키는 것이 전 시간에 진행했던 실습에서 사용했던 내림차순 오름차순 패턴과 비슷하다고 생각하여 사용했던 코드를 가져와 사용했다. 비트 쉬프트 숫자를 바꾸니 엉뚱한게 켜졌다. 더해주는 값을 변경하여 해결하였다.

두 번째 테스트, ISR(인터럽트 서비스 루틴) 함수를 지정하는데 먼저 a 변수를 ISR 함수 밑에 입력하여 인식하지 못하는 오류가 발생했다. 그래서 함수가 시작하기 전 초기값 세팅할 때 a 변수를 입력해주었더니 ISR 함수가 정상 작동하였다.

마지막 테스트, INT0이 발생할 때 한 주기는 정상적으로 작동하였으나 다음 주기를 반복하여 작동하지 못했다. 변수값 next_pattern_value와 reverse_next_pattern_value의 값이 달라졌기 때문이었다. 한 주기가 끝나면 초기값으로 되돌려서 정상 작동하도록 수정했다.

INT0 발생 시 짝수, 홀수 패턴으로 2칸씩 쉬프트 동작하는 것을 반복하였고, INT1은 성공적으로 동작을 멈추었다.

7. 결과에 대한 논의

아쉬웠던 점은 저번에 사용했던 코드를 사용하며 비교할 수 있도록 이름을 바꾸지 않았던 점과 변수를 사용한다고 가정을 할 땐 사용하려는 함수 위에서 정의해야 한다는 것을 잊고 진행했던 점이 아쉬웠다.

이번 실습에서는 아쉬운 점보다 좋아진 점이 많았다.

저번 실습에서 코딩에 대한 지식이 전무해서 실습에서 큰 도움이 되지 못했던 것이 가슴에 남았었다.
실습이 끝난 후 비트 연산자 및 코드 작동 방식에 대해 공부하고 온 뒤에 이번 실습을 진행하니 감회가 새로웠다.

그렇게 관심을 가지고 실험에 대해 탐구를 한 결과, 한 번 사용해봤던 비트 쉬프트 연산자와 새롭게 배운 ISR 함수를 사용하는 법도 비교적 빨리 익힐 수 있었다. 인터럽트 동작 중 다른 인터럽트를 발생시키려면 ISR 함수가 메인 프로그램에서 동작했어야 하는 사실이 새로웠고, 이러한 인터럽트를 통해서 하나의 프로그램이 여러 가지 동작을 할 수 있다는 것도 깨닫게 되었다.

실습을 하며 처음으로 빨리 마쳐봤던 시간이라 더욱 인상 깊었다. 앞으로도 열심히 공부하여 실습에서 빠르게 적용할 수 있도록 노력해야겠다는 다짐도 하게 된다.
앞으로 어떤 함수와 코드를 통해 다양한 동작을 시킬 수 있을지 더욱 기대되는 부분이다.

8. 결론

ATmega128 마이크로 컨트롤러 보드가 Interrupt 함수를 통해 메인 프로그램이 작동하는 와중에 인터럽트를 발생시켜 다른 동작을 처리하고 돌아올 수 있음에 대해 배울 수 있었다. 이러한 동작을 통해 한 가지만 할 줄 아는 프로그램이 아닌 다중 동작을 할 수 있는 프로그램을 만든다는 것을 알 수 있었다.

9. 참고문헌 및 출처

(교재) 김도인 교수님, 「실습교재6」, 강원대학교, 전기전자공학과, (2~11p)
(교재) 김도인 교수님 [2024] 마프_Part 06_인터럽트_ver4, “, ”, (5~26p)
(그림과 사진) 이동제, 실습 중 찍은 사진, 직접 그린 그림, 2024-05-02~05-09